

Using Data Science Techniques In Cybersecurity - Project Report

Dolev Zohar, 330769092

Link to github:

github.com/DolevZohar/UniUsingCyberDataScienceInCybersecurity

Link to the original paper: <https://arxiv.org/pdf/2603.27326>

1. Summary of the Source

In the project I have chosen the paper: Context-Aware Phishing Email Detection Using Machine Learning and NLP. As is apparent from the title of the paper, the main problem I will be looking at throughout this project is Phishing detection. Phishing is one of the most common attacks, and remains prevalent today, as it is simple for attackers to perform, profitable and can be used against both individuals and organizations. It is also worth noting that in light of the rise of LLMs in recent years phishing attempts are gradually becoming more and more convincing, creating a demand for a good and efficient phishing detection system.

The writers of the paper have decided to create a phishing detection system based on the email body. According to the paper, other approaches, such as rule based filtering and url filtering come short. This happens as rule based filtering systems need to be constantly updated with new rules, and url detection might fail if the attackers avoid using obviously suspicious links. Thus, the paper aims to propose a context-aware system, that based on the content of the email comes to a conclusion and classifies an email as either safe or phishing. The goal is to detect general contextual patterns, such as urgency, suspicious wordings and calls to action.

The authors train their system using three publicly available datasets, which are combined into one corpus made up of 53,973 labeled emails, of which 22,035 are phishing and 31,938 are safe (This figure is flipped at a certain point in the paper, but later on is corrected). Dataset 1 contains 11,322 safe emails and 7,312 phishing emails, Dataset 2 is originally a spam-versus-ham dataset, and contains 4,825 ham emails and 747 spam emails. This is mapped in the paper as spam being phishing and ham being safe. Dataset 3 contains 15,791 safe emails and 13,976 phishing emails.

Lastly, the methodology used by the authors includes a NLP preprocessing pipeline, together with Logistic Regression and Multinomial Naive Bayes models. The preprocessing pipeline consists of HTML tag removal, URL removal, Email address removal, numeric removal, special character removal, whitespace normalization,

lowercasing, tokenization, stop word removal, and finally lemmatization, which means words are reduced to their base forms.

Following that, feature extraction is done using TF-IDF vectorizer, which scores the words based on how frequent they are in the document, and how rare they are across the whole dataset. This feature extraction is limited to 5000 features and includes both unigrams and bigrams. The authors also removed terms that appear in less than 2 documents, and terms that appear in more than 95% of documents. This allows the model to learn individual words, as well as short phrases that might be useful for phishing detection.

After that, the paper evaluates the 2 models, Multinomial Naive Bayes and Logistic Regression. Naive Bayes is used because it is efficient and commonly performs well on sparse text classification problems, while Logistic Regression is used because it can model feature correlation more directly and its learned coefficients can be inspected for interpretability. The authors train using a stratified 80/20 train-test split.

The result reported in the paper shows that Logistic Regression outperformed Multinomial Naive Bayes. The paper reports that Logistic Regression achieved an accuracy of 95.41% and an F1 score of 0.9433, outperforming Naive Bayes by 1.5% in accuracy and 0.02 in F1 score. It is worth noting that the models achieved similar Recalls, but Logistic Regression improved the precision compared to Naive Bayes, jumping from 0.918 to 0.95. This means that Logistic Regression reduces the false positives rate, which is impactful for actual deployment, as false positives can disrupt natural communications.

In addition, the paper also presents a deployed web-based phishing detection system where the Logistic Regression model is used. This interface allows users to submit emails and receive a classification in real time.

2. Critical Evaluation

The paper's main claim is that a context-aware phishing email detection system based on NLP and classical machine learning can achieve strong, practical and interpretable performance. More specifically, the authors claim that Logistic Regression trained on TF-IDF features can outperform Multinomial Naive Bayes and is suitable for real time phishing classification.

The narrower claim of Logistic Regression outperforming Naive Bayes is clearly supported by both the results shown in the paper and the results I have got in the reproduction of the model in the submitted notebook.

The evaluation methodology consisted of testing on part of the combined dataset, that was omitted for testing purposes in the 80/20 stratified train-test split. The results were later inspected with the metrics of accuracy, recall, precision and F1 score. These metrics are useful to us, as accuracy tells us how often the model is

correct in general, precision gives us information about false positives and recall about false negatives, while F1 score takes both of these into account. In this project, MCC, ROC-AUC, PR-AUC, and F2-score were also added to provide a broader evaluation of model performance.

However, even in the paper itself the authors show that some of the words that got the highest safe coefficients are dataset specific phrases, such as 'enron', 'vince' and 'houston'. These are clearly not tells of an actual safe email, but rather the model learning the dataset itself instead of the general classification task. Furthermore, during some of my testing, which I will go into more later, when I tested training the model on 2 of the datasets and testing on the third the results dropped substantially, which further put in question the generalization ability of the model. In fact, the results of testing on the third dataset ranged from a drop to 90% accuracy all the way to 40%, which indicates that the model failed to generalize the learned data to general phishing detection. The models were, however, stable across random partitions of the same combined corpus, but this did not translate into reliable generalization to unseen dataset sources.

Another point to notice is that the model uses only unigrams and bigrams, which means it can only look at singular words and short 2 word phrases. This limits the context window, which is the focal point of the paper. Finally, the paper also discusses error analysis, where it is shown that due to relying only on email content, without any metadata, URLs or emails, general promotional emails get classified as phishing emails with very high confidence. This result also appeared in the reproduction of the model in the notebook.

Especially in cases like this, but even in general phishing detection cases, the metadata of the email, as well as the information removed during preprocessing such as URLs and emails can help the model reach a better classifying decision.

Overall, the results suggest that the model has limited generalization beyond the learned datasets, and seems to have at least partially learned a lot of the dataset rather than the general phishing detection task. This, along with the fact that the model can't distinguish between promotional emails and phishing emails mean it is not practical and can't be used commonly as a phishing detection system without changes or improvements.

3. Feature Engineering Analysis

Feature engineering was critical to the paper's methodology. Since the raw input is email text, the model can't use that directly. The main goal of the feature engineering

process here is to transform the unstructured email bodies into numerical features that can be used by the model.

The preprocessing pipeline in the paper removes HTML tags, URLs, email addresses, numbers, and special characters. After that, it applies lowercasing, tokenization, stop-word removal, and lemmatization. These steps are used to reduce noise in the raw email text and to consolidate different forms of the same word. For example, "Account," "account," and "accounts" can all be reduced to the same base form after lowercasing and lemmatization.

The removal steps can be justified because the model is designed as a content-based linguistic classifier. HTML tags, special characters, long URLs, email addresses, and many numeric values can create sparse and inconsistent tokens that may not generalize well. Since the model uses TF-IDF with unigrams and bigrams, it does not truly analyze URL structure, sender reputation, domain similarity, or HTML formatting. Therefore, keeping raw URLs or email addresses as ordinary text tokens may not help the model use them correctly.

TF-IDF vectorizer is then used and also performs feature selection. The model keeps a maximum of 5,000 features, removes terms that appear in only one document and terms that appear in more than 95% of the documents. This helps us filter terms that might be very specific to one specific document, but not help us in general, and also terms that are too global, and do not offer any further information. Limiting the number of features is important to us as text data naturally creates a very high dimension feature space, while reducing the vocabulary makes the model faster, less noisy and easier to interpret.

From a cybersecurity perspective, this feature engineering process is meaningful because phishing emails often rely on recognizable language patterns, such as words or phrases relating to clicking links, verifying accounts and general urgent messaging and calls to action. However, this approach also has a limitation as it may learn dataset specific artifacts, as we discussed in the previous section, Critical Evaluation.

It is also important to note that there is some redundancy in the feature space, as the vectorizer uses both unigrams and bigrams, and a phrase and the individual words that make it up often carry similar information. This redundancy is not necessarily harmful, as it can help us in certain cases and allows the model to have more context, but it is something to keep note of and something that might cause the model to learn more artifacts of the dataset.

The preprocessing also creates a clear weakness in removing potentially useful information. URLs, email addresses, numbers and HTML structure may contain important phishing indicators. For example suspicious domains, unusual link

structure or heavy HTML formatting can all be useful in phishing detection. Regardless, the authors of the paper decided to remove these in order to focus on the linguistic information in the email, but a more complete system that can also analyze URLs, email addresses and additional information could make use of the removed data.

In general, the feature engineering process is appropriate for a classical NLP baseline. It produces interpretable and efficient text features and helps the Logistic Regression model perform well on the combined dataset. The approach is still limited however as it omits a lot of potentially useful information. There are also additional features that could be used such as sender metadata and attachment information that could help the model, as often we understand that an email is suspicious based on a suspicious sender or attachment.

4. Reproducibility Analysis

The paper is mostly reproducible. While the github repository linked in the paper doesn't contain the pipeline showcased in the paper, from loading the datasets, to training the model and testing it the paper itself still shows enough information to reproduce the experiments. In particular it uses publicly available datasets, and writes clearly all of the parameters used for the Naive Bayes and Logistic Regression models, as well as the TF-IDF vectorizer.

I briefly mentioned that the github repository doesn't contain the pipeline. It instead contains the final interface mentioned in the paper, and contains a requirements file that can be used to download the dependencies. This interface however, is not the main focus of the project. It is also worth noting that in this interface the authors also use rule based decision making together with the model as a way to boost its ability.

It is also worth noting that there seems to be another hidden preprocessing step that is not mentioned in the paper. In particular after combining the datasets I have noticed I have 13 more samples, which is a small change. However, after noticing there were some exceedingly long emails I filtered all samples with length over 20,000 words, which got the size of the dataset to equal exactly the reported size in the paper. It is still unknown however if that was truly the hidden step and the difference.

Overall, the model itself is reproducible, and the main pipeline can be recreated while the datasets are publicly available. This is supported by the fact that the reproduction got results that are very close to the results stated in the paper. The small differences could be caused by small hidden steps that were used and unreported in the paper.

5. Experimental Results

After reproducing the results of the paper I moved on to original experiments in order to test additional options and the performance of the model.

There were 7 experiments done:

- Keeping the URLs instead of removing them in the preprocessing.
- Heldout dataset testing, where we train on 2 datasets and test on the third.
- Testing additional classifiers, XGBoost and Random Forest, and comparing them to the Logistic Regression model.
- Extending the context window of the model and adding trigrams.
- Testing the model trained only on datasets 1 and 3, without dataset 2 that was originally a spam-versus-ham dataset.
- Cross-validation testing, meant to test whether the results stay consistent using different partitions of the dataset.
- Choosing a different decision threshold by maximizing F2-score on a validation set

For these experiments, the metrics used were accuracy, precision, recall, F1-score, F2-score, MCC, ROC-AUC, and PR-AUC. MCC, ROC-AUC, PR-AUC, and F2-score were added to the metrics used in the original paper, with each providing different information about model performance. MCC uses all four confusion-matrix outcomes, taking into account true positives, false positives, false negatives, and true negatives. ROC-AUC measures how well the model ranks phishing emails above safe emails across different decision thresholds, while PR-AUC summarizes the trade-off between phishing precision and recall. F2-score combines precision and recall while giving greater importance to recall. PR-AUC is useful for evaluating performance on the phishing class, while F2-score explicitly emphasizes reducing false negatives that hold greater importance to us compared to false positives.

I will now go over the idea behind each experiment, the experiment itself, the results and conclusion.

Firstly, I tested whether keeping the URLs in the raw text is helpful. As we know, URLs can be very helpful for phishing detection as often a suspicious URL is the main indicator of phishing. For this experiment I have loaded the same combined dataset and have used the same preprocessing of the main model in the paper, except that I have used 3 variants, one where the URLs were removed, one where they were kept as they are and an additional one replacing URLs with a URL token. The results were underwhelming, as all 3 models got very close results. This perhaps means that with the model's limited understanding the addition of URLs are not helpful as it can't analyze the URLs and use them to determine if the email is a phishing email or not. This means that in the future, should URLs inspection be

added, a larger effort needs to be made to use it, perhaps training a separate model to distinguish between safe and suspicious URLs.

Next, there was the heldout dataset experiment. This was done mainly to test the generalization ability of the model. This is even more important than usual as some of the things we saw, such as the high coefficients of dataset specific terms caused us to doubt the generalization ability of the model.

For this experiment I have trained 3 models of Logistic Regression, each time with a different pair of datasets. The datasets have gone through the preprocessing and vectorization pipeline previously mentioned. After training on 2 datasets I have tested each model on the dataset it was not trained on.

The results were far from ideal, in every case the metrics dropped. In the best case, when testing on the enron dataset, which seemed to be the one the model was learning the most, the results weren't all too bad, achieving 93% accuracy with a high 0.98 precision. Recall however, dropped to 0.86, showing there were more false negatives. ROC-AUC however, gave a good metric of 0.99 which may suggest that with a different decision threshold we can get better results.

The other cases weren't as good however. In particular, when testing on dataset 2 we achieved an accuracy of 42%, with precision dropping all the way to 0.17. In fact, most of the test set was classified as a false positive.

There are 3 main conclusions to take from this experiment. The first is that the enron dataset seems to cause the model to learn its artifacts specifically instead of the general task. The second conclusion is that dataset 2, which is originally a spam-ham dataset and not a phishing dataset might not fit in well with the task we are trying to train the model for. This will be further tested in experiment 3 where we test the model trained without dataset 2.

Lastly, there is also the obvious conclusion that the model's generalization is under question, and we can't be sure of its generalization ability just by looking at the metrics reported in the paper.

In the third experiment I tested the additional classifiers XGBoost and Random Forest. The paper mentioned that Logistic Regression can compete with more 'advanced' techniques and mentions a previous work where Random Forest was used and got 1.5% worse accuracy than the results of the paper.

I trained XGBoost, Random Forest and Logistic Regression and Naive Bayes all on the same combined dataset that has been through the previously mentioned pipeline.

The results have shown that all 3 models outperform Naive Bayes, with Logistic Regression and Random Forest outperforming XGBoost. Logistic Regression and Random Forest have gotten very close accuracy, MCC, ROC-AUC and F1 scores, differing mostly in the tradeoff between false positives and false negatives, with Random Forest getting less false positives in exchange for slightly more false negatives.

While the additional classifiers failed to improve significantly upon the results of Logistic Regression Random Forest still outperformed the supposed previous experiment that was done on a much smaller dataset (the dataset used in this experiment is approximately 18 times larger).

It is also worth noting that it is possible that with better feature engineering for the models, it could be possible to achieve better results.

In the fourth experiment, I've tried extending the context window of the model, as the focal point of the paper is that the model is context aware. To do that I've trained Naive Bayes and Logistic Regression both on the same dataset with the same preprocessing. The first set of models was trained after TF-IDF vectorization with only unigrams and bigrams and the second set with trigrams as well. In order to keep up with the rising amount of n-grams, I've also raised the max features to 10,000. The results have shown that the addition of trigrams improved the models, improving all metrics, although the improvement wasn't very high, raising Logistic Regression's accuracy from 95% to 96% and F1 score from 0.94 to 0.95, lowering both false positives and false negatives. It is also worth noting that Naive Bayes remained outclassed in this experiment.

While the addition of trigrams improved the metrics of the model, it is also important to keep in mind that it is possible that the addition of larger n-grams will cause the model to overfit, thus it is a tradeoff and most likely shouldn't be expanded on to larger n-grams without doing further changes to the model.

Experiment 5 consisted of testing the model trained only on datasets 1 and 3. As we mentioned previously, dataset 2 was originally a spam-versus-ham dataset, and the authors of the paper treated it as a phishing-versus-safe dataset for the purposes of the phishing detection task. This experiment is meant to test whether or not this decision was plausible, and to check if the spam-versus-ham dataset fits in well with the other 2 datasets.

This experiment includes both Multinomial Naive Bayes as well as Logistic Regression. The preprocessing pipeline and vectorization are the same as the one used in the paper, and we train 2 sets of models. The first trained only on datasets 1 and 3 and the other trained on all 3 datasets, used for comparison. The results show a clear rise in almost all metrics. In particular Logistic Regression, which had better results compared to Naive Bayes, showed an increase in accuracy from 0.956 to 0.974, recall increased from 0.94 to 0.983, F2-score increased from 0.943 to 0.978 and the false negative count dropped from 262 to 69, decreasing by 73% while the test size decreased by 10.3% only. This directly translates to a substantial drop in false negative rate, from 5.95% to 1.62%.

These results suggest that spam-versus-ham classification should not automatically be treated as equivalent to phishing-versus-safe classification, since including Dataset 2 reduced performance in this experimental setting. In fact, removing the out

of place spam-versus-ham dataset improved most metrics substantially, only seeing a small increase in false positive rate, from 3.18% to 3.27%.

However, it must also be noted that the increase in results might also stem from the possibility that the removal of the second dataset made the resulting dataset more homogeneous and allowed the model to more closely learn it and its artifacts.

The sixth experiment consisted of repeated stratified cross validation, using different dataset partitions. The main focus of the experiment was to test whether the results remain consistent across different data partitions. The models used in the experiment are Naive Bayes and Logistic Regression. Each model was evaluated using both the complete dataset and a version containing only Datasets 1 and 3, following up on the positive results obtained in Experiment 5.

The preprocessing pipeline and vectorization settings remained unchanged, while the training was performed using repeated stratified five-fold cross-validation with 3 repetitions, producing 15 evaluations for each of the 4 model and dataset configurations. In every fold, the model and vectorizer were only fitted on the training set to avoid data leakage.

The results showed small standard deviations across all four configurations, indicating that model performance was highly consistent across different stratified data partitions. The mean results were also close to those obtained from the original train-test split, suggesting that the original split was reasonably representative of the combined corpus.

While this is a positive result, it cannot be translated directly into a conclusion about the models' ability to generalize to unseen dataset sources, because the training and validation folds still contain emails drawn from the same dataset mixture.

Lastly, the seventh experiment tested whether the default decision threshold of 50% probability was appropriate for our purposes. The default threshold of 0.5 is a conventional choice, but it is not necessarily aligned with our greater concern about false negatives.

That is why in this experiment we've tried to find a better threshold. This was done by creating a train/validation/test split of 60/20/20, with the goal of training on the training set, finding the best decision threshold by maximizing F2-score on the validation set and finally testing on the test set. F2-score was chosen as the metric to maximize since it takes into account both precision and recall, while explicitly giving greater importance to recall.

It is also important to note that since previous experiments have shown that a model trained only on datasets 1 and 3 achieves better results, the model used in this experiment also excludes dataset 2.

Following that, after using the validation set the selected threshold was 36.98%. This means that any predicted phishing probability at or above that threshold will be

classified as phishing. This takes into account our preference, as generally in our case false negatives are more harmful as they mean that a harmful phishing email has reached a user. It also aligns with real world logic where we don't always need to be sure, we just need to have sufficient suspicion. Using this threshold recall rose from 0.9833 to 0.9915, F2-score rose from 0.9787 to 0.9808 while precision dropped from 0.9605 to 0.9403. The false positive and false negative counts also highlight these results, with false positives rising from 172 to 268 and false negatives decreasing from 71 to 36. These results are to be expected, since we optimized the threshold for F2-score which emphasizes recall over precision. However, F2-score still takes precision into account. Thresholds below 36.98% produced lower validation F2-scores because the additional loss in precision was too large to be offset by the improvement in recall, even though recall receives greater weight in F2-score.

Overall, this experiment demonstrates the tradeoff between precision and recall when it comes to the decision threshold, and the fact there isn't an absolute threshold, but rather that the threshold needs to be chosen based on the needs of the system.

6. Conclusions

The main finding of this project is that the paper's central experiment result is mostly reproducible. Logistic Regression with TF-IDF features performed better than Multinomial Naive Bayes on the combined phishing email dataset, which supports the claims of the authors. However, the additional experiments showed that this result should be interpreted carefully, as during the heldout dataset testing we have seen that when the model doesn't train on the specific dataset the performance drops substantially.

We have also found that the choice of datasets might have been wrong, as the enron dataset contains too many dataset specific terms, and the spam-ham dataset seems to be different from the task we are trying to achieve.

Additionally, excluding the spam-versus-ham dataset improved most evaluation metrics. However, this may partly be because the remaining dataset became more homogeneous and easier to classify, so the improvement should not be interpreted as evidence of better generalization.

The main lesson learned is that high performance on a random train-test split is not enough to prove that a cybersecurity model generalizes well and will work well in realistic conditions. In phishing detection generalization is also very important as phishing campaigns, writing styles, domain and attack strategies constantly change.

This project also showed the importance of feature importance analysis and error analysis as they helped reveal helpful dataset bias and practical failure cases such as promotional material.

However, the proposed solution still has several strengths. It is simple, efficient, interpretable and easily reproducible. TF-IDF and Logistic Regression are capable of creating a strong baseline, and the model's coefficients can be inspected to get a better understanding of which words or phrases influence the classification.

Furthermore, repeated cross-validation testing has shown that the model's results don't rely on a specific favorable data partition, but rather that the results are consistent across different partitions.

The model also has important weaknesses. It relies on surface level patterns, uses only unigrams and bigrams and removes a lot of potentially useful information such as URLs, email addresses and doesn't take into account sender information. Other than that, it also seems to have limited generalization ability across datasets.

Personally, I think future improvements should focus on combining multiple approaches. For example, using a model that focuses only on the identity of the sender, together with a model that tries to classify whether or not a link is suspicious together with a model that looks at the content of the email can achieve better results. Also, as we've seen in one of the experiments, the decision threshold is not a universal value, and should be selected based on the needs of the system. Other than that, in light of what we have seen with this model, evaluation should also include testing on data from unseen datasets, as with this it will be easier to see whether the model learns the task as intended or instead learns dataset artifacts.